



Complete Study Material

Architecture

Integration

Security

Real-World Use Cases

10 Topics · Code Examples · Architecture Diagrams · Best Practices

Prepared for: MCP Learning Series | 2025

TABLE OF CONTENTS

01

What is MCP (Model Context Protocol)?

Introduction & Core Concepts

02

Why MCP Matters

Benefits & Value Proposition

03

MCP Architecture

Servers, Clients & Transports

04

MCP vs Traditional APIs

Comparison & Decision Guide

05

MCP Server Types

File, Database & Web Servers

06

Setting up MCP

Installation & Configuration

07

Building an MCP Server

Step-by-Step Development

08

MCP with Claude Integration

Connecting Claude to MCP

09

Real-world MCP Use Cases

Industry Applications

10

MCP Best Practices & Security

Hardening & Guidelines

01 What is MCP (Model Context Protocol)?

The **Model Context Protocol (MCP)** is an open standard developed by Anthropic that defines how AI models communicate with external tools, data sources, and services. It provides a unified, language-agnostic interface so that any AI application can plug into any data source or capability without writing custom integrations for every combination.

■ MCP is to AI integrations what USB is to devices — one standard connector for everything.

Before MCP, connecting an AI assistant to a database, a file system, or a web API required bespoke adapters written in isolation. MCP abstracts this problem by defining a protocol — a shared language — between the **AI host** (like Claude Desktop) and **MCP servers** that expose capabilities.

■ Core Concepts

Term	Definition
Protocol	A formal set of rules for how messages are structured and exchanged between clients and servers.
Context	Information that helps the model understand its environment — file contents, database records, API results, etc.
Host	The application that embeds the AI model (e.g., Claude Desktop, VS Code extension).
Client	The component inside the host that speaks MCP and connects to servers.
Server	An external process that exposes tools, resources, and prompts via the MCP protocol.

■ Key Capabilities Exposed via MCP

- Tools — functions the AI can call (e.g., run a SQL query, send an email)
- Resources — data sources the AI can read (e.g., files, database rows)
- Prompts — reusable prompt templates registered on the server
- Sampling — the server can ask the client to run AI inference

02 Why MCP Matters

Without a standard protocol every AI product integration becomes a one-off project. Teams rewrite the same glue code repeatedly, security reviews are inconsistent, and adding a new tool means engineering effort every single time. MCP solves this.

■ Key Benefits

■ Plug-and-play	Any MCP-compatible client can use any MCP server instantly — no custom integration code needed.
■ Secure by design	Fine-grained permission scoping per server. The AI only sees what it's allowed to see.
■ Reusable servers	A single MCP server for Postgres can serve Claude Desktop, VS Code, a Slack bot, and a custom app simultaneously.
■ Reduced latency	Local MCP servers (stdio transport) avoid HTTP round-trips, giving near-zero overhead.
■ Open ecosystem	Anthropic open-sourced the spec. Any vendor can build an MCP server — thousands already exist.
■ Composable	Hosts can connect to multiple servers at once, mixing file, database, and web capabilities freely.

■ The Business Impact

Organisations adopting MCP report dramatically faster time-to-integration for new data sources. Security teams benefit because permissions are declared in one place. Developers can publish an MCP server once and have it work with every AI assistant their organisation uses — today and in the future.

03 MCP Architecture

MCP uses a client–server architecture. The **host application** embeds one or more **MCP clients**, each of which maintains a 1-to-1 connection with a single **MCP server**. Communication travels over a **transport layer**.



■ Components in Detail

Component	Role
Host	The outer application (Claude Desktop, IDEs, chatbots). Manages lifecycle of all clients.
MCP Client	Lives inside the host. Handles one server connection. Translates AI requests into MCP messages.
MCP Server	Standalone process (or remote service) that exposes Tools, Resources, and Prompts.
Transport	The communication channel. Two primary types: stdio (local) and HTTP+SSE (remote).
Protocol Layer	JSON-RPC 2.0 messages with MCP-specific methods: initialize, tools/list, tools/call, resources/read, etc.

■ Transport Types

Transport	Full Name	Description	Use When
stdio	Standard Input/Output	Local processes only. Zero network overhead. Used for file systems, local DBs.	■ Best for local
HTTP + SSE	HTTP with Server-Sent Events	Remote servers over the internet. Supports auth headers, firewalls. Real-time streaming.	■ Best for cloud
WebSockets	Bidirectional socket	Experimental in some implementations. Low-latency bidirectional comms.	■ Emerging

■ Connection Lifecycle

- 1. Host launches the MCP server process (or connects to a remote URL).
- 2. Client sends initialize request with its capabilities.
- 3. Server responds with its capabilities (tools, resources, prompts).
- 4. Client sends initialized notification — connection is ready.
- 5. AI model calls tools/list or resources/list to discover capabilities.
- 6. AI calls tools/call or resources/read as needed during inference.
- 7. Either side can close the connection gracefully.

04 MCP vs Traditional APIs

Traditional REST or GraphQL APIs are designed for application-to-application communication where the calling code is human-written and deterministic. MCP is designed for **AI-to-service communication** where the caller (the model) is autonomous and needs to discover capabilities at runtime.

Aspect	Traditional API	MCP
Capability Discovery	Hard-coded in client code	Dynamic — model lists tools at runtime
Schema	OpenAPI / custom docs	Structured JSON Schema in tools/list
Authentication	Per-API credentials	Declared per-server, managed by host
Error Handling	HTTP status codes	Structured MCP error objects with context
Streaming	Requires SSE/WebSocket setup	Built-in streaming support
Composability	Manual orchestration needed	Multiple servers auto-composed by host
Context Passing	Caller manages all state	Server manages resource state, client requests slices
AI Optimisation	Not designed for LLMs	Descriptions, examples, and annotations for LLMs
Versioning	Breaking changes common	Negotiated during initialize handshake
Local Execution	Network required	stdio transport for zero-network local use

■ When to Choose MCP over a Direct API

- The AI needs to discover what it can do at runtime (dynamic capability listing).
- You want one integration pattern across multiple AI systems.
- You need tight permission control at the capability level.
- The AI will call the same service many times in one session (session-level context).
- You are building a developer-facing AI tooling product (IDE assistants, copilots).

■ Both can coexist: an MCP server can itself call traditional REST APIs internally.

05 MCP Server Types

MCP servers are categorised by the type of data or capability they expose. The three primary categories are File, Database, and Web servers, though any external capability can be wrapped as an MCP server.

■ File System Servers

- Expose local or remote file systems as MCP resources.
- Provide tools: `read_file`, `write_file`, `list_directory`, `search_files`.
- Examples: filesystem MCP server (official Anthropic), Google Drive MCP.
- Use case: letting Claude read your codebase, write reports, edit documents.

■ Database Servers

- Expose SQL or NoSQL databases as queryable resources.
- Provide tools: `execute_query`, `list_tables`, `describe_schema`.
- Examples: Postgres MCP, SQLite MCP, MongoDB MCP.
- Use case: letting Claude run analytics queries, update records safely.

■ Web & API Servers

- Expose web browsing, HTTP APIs, or web scraping capabilities.
- Provide tools: `fetch_url`, `search_web`, `post_request`, `get_request`.
- Examples: Brave Search MCP, Puppeteer MCP, GitHub MCP.
- Use case: letting Claude research, interact with SaaS platforms, open PRs.

■ Other Notable Server Categories

Category	Examples
Communication	Email (Gmail), Slack, Discord, calendar systems
Cloud & DevOps	AWS, GCP, Azure, Docker, Kubernetes
Code & Dev Tools	GitHub, GitLab, Jira, Linear, Sentry
Knowledge Bases	Notion, Obsidian, Confluence, custom RAG stores
Multimedia	Image generation (DALL·E, Stable Diffusion), audio processing
IoT / Hardware	Home automation (Home Assistant), sensor data streams

06 Setting up MCP

Setting up MCP involves installing the SDK for your language, configuring your host application to know about your servers, and verifying the connection. The official SDKs are available for **TypeScript/JavaScript** and **Python**.

■ Prerequisites

- Node.js 18+ (for TypeScript SDK) or Python 3.10+ (for Python SDK)
- Claude Desktop app (for local testing) or any MCP-compatible host
- npm or pip package manager

■ Installation

```
# Install MCP SDK

# TypeScript / JavaScript

npm install @modelcontextprotocol/sdk

# Python

pip install mcp

# Or with uv (recommended for Python)

uv add mcp
```

■ Claude Desktop Configuration

To connect Claude Desktop to an MCP server, edit the configuration file at **~/Library/Application Support/Claude/claude_desktop_config.json** (macOS) or **%APPDATA%\Claude\claude_desktop_config.json** (Windows):

```
# claude_desktop_config.json

{

  "mcpServers": {

    "filesystem": {
```

```
"command": "npx",

"args": ["-y", "@modelcontextprotocol/server-filesystem", "/home/user/docs"]

},

"postgres": {

"command": "npx",

"args": ["-y", "@modelcontextprotocol/server-postgres"],

"env": { "DATABASE_URL": "postgresql://localhost/mydb" }

}

}

}
```

■ Verifying the Connection

- Restart Claude Desktop after saving the config.
- Open a new chat and click the ■ (plug) icon in the toolbar.
- You should see your servers listed with a green connected indicator.
- Ask Claude: 'What tools do you have available?' to verify discovery.

■ Use 'npx @modelcontextprotocol/inspector' to debug MCP server connections visually.

07 Building an MCP Server

Building an MCP server means writing a process that speaks the MCP protocol. The SDK handles all the wire-format details; you just declare your tools and resources.

■ Python Example — Weather Tool Server

```
# weather_server.py

from mcp.server import Server

from mcp.server.stdio import stdio_server

from mcp import types

import httpx, asyncio

server = Server('weather-server')

@server.list_tools()

async def list_tools() -> list[types.Tool]:

    return [

        types.Tool(

            name='get_weather',

            description='Fetch current weather for a city',

            inputSchema={

                'type': 'object',

                'properties': {

                    'city': {'type': 'string', 'description': 'City name'}

                },

                'required': ['city']
```

```

}

)

]

@server.call_tool()

async def call_tool(name: str, arguments: dict):

    if name == 'get_weather':

        city = arguments['city']

        async with httpx.AsyncClient() as client:

            r = await client.get(f'https://wttr.in/{city}?format=j1')

            data = r.json()['current_condition'][0]

            return [types.TextContent(

                type='text',

                text=f"{city}: {data['temp_C']}°C, {data['weatherDesc'][0]['value']}"

            )]

        async def main():

            async with stdio_server() as streams:

                await server.run(*streams, server.create_initialization_options())

        if __name__ == '__main__':

            asyncio.run(main())

```

■ TypeScript Example — Notes Resource Server

```

# notes_server.ts

import { Server } from '@modelcontextprotocol/sdk/server/index.js';

```

```

import { StdioServerTransport } from '@modelcontextprotocol/sdk/server/stdio.js';

const server = new Server({ name: 'notes-server', version: '1.0.0' },
{ capabilities: { resources: {}, tools: {} } }

);

const notes = new Map();

server.setRequestHandler(ListResourcesRequestSchema, async () => ({
resources: [...notes.keys()].map(id => ({
uri: `note:///${id}`, name: id, mimeType: 'text/plain'
}))
}));

server.setRequestHandler(ReadResourceRequestSchema, async (req) => {
const id = req.params.uri.replace('note://', '');
return { contents: [{ uri: req.params.uri, text: notes.get(id) ?? '' } ] };
});

const transport = new StdioServerTransport();
await server.connect(transport);

```

■ Server Development Checklist

- Define clear, LLM-friendly tool descriptions (the model reads these).
- Use strict JSON Schema for inputSchema to prevent invalid calls.
- Return structured errors with descriptive messages, not raw exceptions.
- Test with the MCP Inspector before connecting to Claude.
- Add resource pagination for large datasets (cursor-based).
- Log all tool calls and results for debugging and auditing.

08 MCP with Claude Integration

Claude supports MCP natively in Claude Desktop and via the Anthropic API when building custom applications. This section covers both integration paths.

■ Path 1: Claude Desktop (No-Code)

- Install Claude Desktop from claude.ai/download.
- Edit `claude_desktop_config.json` to declare your servers (see Section 06).
- Restart Claude Desktop — servers are connected automatically.
- Claude will use available tools automatically when relevant, or you can ask it to.

■ Path 2: Anthropic API + MCP (Custom Apps)

When building a custom AI application you can use the **MCP client libraries** to connect to servers, then pass the discovered tool schemas to the Claude API as standard tool-use parameters.

```
# claude_mcp_integration.py

import anthropic

from mcp import ClientSession, StdioServerParameters

from mcp.client.stdio import stdio_client

async def run_with_mcp():

    # 1. Connect to MCP server

    params = StdioServerParameters(command='python', args=['my_server.py'])

    async with stdio_client(params) as (read, write):

        async with ClientSession(read, write) as session:

            await session.initialize()

            # 2. Get tool list from MCP server

            mcp_tools = await session.list_tools()

            # 3. Convert to Anthropic tool format
```

```

tools = [{'name': t.name, 'description': t.description,
          'input_schema': t.inputSchema}

for t in mcp_tools.tools]

# 4. Call Claude with MCP tools

client = anthropic.Anthropic()

response = client.messages.create(

model='claude-opus-4-5',

max_tokens=1024,

tools=tools,

messages=[{'role': 'user', 'content': 'What tools can you use?'}]

)

# 5. Execute tool calls via MCP

for block in response.content:

if block.type == 'tool_use':

result = await session.call_tool(block.name, block.input)

print(result)

```

■ claude.ai Hosted MCP (Remote Servers)

Claude.ai also supports connecting to **remote MCP servers** via HTTP+SSE. Organisations can host MCP servers on their infrastructure and connect them to Claude.ai for team-wide access, with OAuth-based authentication.

■ Remote MCP servers require HTTPS and support OAuth 2.0 for secure authentication.

09 Real-world MCP Use Cases

MCP is already powering production use cases across industries. Here are ten high-impact applications with implementation notes.

01 Code Review Assistant

Connect GitHub MCP + file system MCP to let Claude read PRs, run linters, check test coverage, and post review comments automatically.

02 Enterprise Data Analyst

Postgres/Snowflake MCP gives Claude read-only access to your data warehouse. Ask questions in plain English and get SQL, charts, and insights back.

03 Customer Support Copilot

Salesforce MCP + knowledge base MCP lets support agents ask Claude for context on any ticket, suggested replies, and escalation guidance.

04 DevOps Incident Response

PagerDuty + CloudWatch + Kubernetes MCP. Claude can read alerts, check pod logs, and suggest remediation steps during an incident.

05 Document Automation

File system + Google Docs MCP. Claude reads templates, fills them with data from a database MCP, and saves the results — fully automated.

06 Research Assistant

Brave Search + arXiv + Zotero MCP. Claude searches papers, reads abstracts, extracts citations, and builds a literature review.

07 Personal Finance Manager

Banking API MCP + spreadsheet MCP. Categorise transactions, track budgets, forecast expenses, and generate monthly reports.

08 Software Onboarding

Codebase file MCP + Confluence docs MCP. Claude answers 'How does X work?' with live code references and team documentation.

09 E-commerce Operations

Shopify + inventory DB MCP. Claude identifies low-stock products, drafts reorder emails, and updates product descriptions.

Legal Document Review

Contract file MCP + legal knowledge base MCP. Claude flags non-standard clauses, compares against standard templates, produces summary reports.

10 MCP Best Practices & Security

MCP grants AI models real-world capabilities. Security is not optional. Apply the following practices to every production MCP deployment.

■ Principle of Least Privilege

Each MCP server should expose the **minimum set of capabilities** required for its purpose. A customer support bot does not need write access to the database.

- Use read-only database connections unless writes are explicitly required.
- Scope file system servers to specific directories, never the root filesystem.
- Use separate API keys per MCP server with minimal scopes.
- Rotate credentials regularly and store them in environment variables, not code.

■ Input Validation & Sanitisation

```
# input_validation.py

# Always validate tool inputs before executing

@server.call_tool()

async def call_tool(name: str, arguments: dict):

    if name == 'execute_query':

        query = arguments.get('query', '')

        # Reject dangerous patterns

        forbidden = ['DROP', 'DELETE', 'TRUNCATE', 'INSERT', 'UPDATE']

        if any(kw in query.upper() for kw in forbidden):

            raise ValueError('Only SELECT queries are permitted')

        # Use parameterised queries – never string concatenation

        return await db.fetch(query) # db handles parameterisation
```

■ Authentication for Remote Servers

Auth Method	When to Use
-------------	-------------

OAuth 2.0	Recommended for user-facing remote servers. Supports token refresh.
API Keys (Header)	Suitable for server-to-server. Always use HTTPS. Rotate regularly.
mTLS	Highest security. Both client and server present certificates. Use for internal services.
stdio (local)	No network auth needed. Security = OS process isolation.

■ Prompt Injection Defence

A malicious document read through a file MCP server could contain instructions designed to hijack the model's behaviour. Mitigate this with:

- Treat all data returned by MCP tools as untrusted content.
- Use system prompt boundaries to separate tool results from instructions.
- Log all tool calls and inputs for post-hoc audit trails.
- Rate-limit tool calls per session to prevent runaway agents.
- Require human-in-the-loop confirmation for high-impact operations (file deletion, financial transactions).

■ Monitoring & Observability

- Log every tool invocation with timestamp, tool name, arguments, and result.
- Alert on anomalous call volumes (could indicate prompt injection loops).
- Track latency per tool — slow tools degrade agent response quality.
- Use structured logging (JSON) for easy ingestion into SIEM tools.

■ Security Checklist

■ Least privilege	Minimal permissions per server
■ Input validation	Reject/sanitise all tool inputs
■ HTTPS everywhere	TLS 1.2+ for all remote transports
■ Secret management	Env vars or secret manager, never hardcoded
■ Audit logging	All tool calls logged and retained
■ Rate limiting	Per-session tool call limits
■ Human approval	For destructive or financial actions
■ Dependency scanning	Keep MCP SDK and dependencies updated

■ Review the official MCP security guidelines at modelcontextprotocol.io/docs/security before any production deployment.

SUMMARY & QUICK REFERENCE

Topic	Key Takeaway
What is MCP?	Open standard for AI ↔ tool communication (JSON-RPC 2.0 based).
Why MCP?	Plug-and-play integrations, security scoping, reusable servers.
Architecture	Host → MCP Client → Transport → MCP Server.
Transports	stdio (local, zero-latency) or HTTP+SSE (remote, cloud).
Server Types	File, Database, Web — plus Communication, DevOps, Code, etc.
Setup	Install SDK, configure <code>claude_desktop_config.json</code> , restart host.
Build Server	Declare tools/resources with JSON Schema; SDK handles protocol.
Claude Integration	Desktop (config file) or API (convert MCP tools → Claude tools).
Use Cases	Code review, data analysis, document automation, incident response...
Security	Least privilege, input validation, audit logging, HTTPS, rate limiting.

Official documentation: modelcontextprotocol.io | SDK source: github.com/modelcontextprotocol | Inspector: `npx @modelcontextprotocol/inspector`